

A Mini Project Report on

VIBE WAVE

Submitted By

YUGESH A

(Reg. No: 242237)

In Partial fulfillment of requirements of the award of Degree of

B.Sc., Computer Science

(Data Science and Analytics)

Under the guidance of

MRS. S. YOGESHWARI M.Sc.



DEPARTMENT OF COMPUTER SCIENCE

SUBBALAKSHMI LAKSHMIPATHY COLLEGE OF SCIENCE

(An Autonomous and ISO 9001:2015 Certified Institution)

Affiliated to Madurai Kamaraj University

Re-accredited with 'A+' Status by NAAC

TVR NAGAR, ARUPPUKOTTAI ROAD

MADURAI-625002

2024-2025



SUBBALAKSHMI LAKSHMIPATHY COLLEGE OF SCIENCE

TVR NAGAR, ARUPPUKOTTAI ROAD, MADURAI-625 022

DEPARTMENT OF COMPUTER SCIENCE

BONAFIDE CERTIFICATE

This is to certify that the project report entitled Vibe Wave is a Bonafide record of the project work done by

Name: Yugesh A

Reg No: 242237

Course & Year: I B.Sc. CS (DSA)

In partial fulfilment of requirement for the award of degree of B.Sc., Computer Science (Data Science and Analytics) and to certify that the project work carried out by him is an independent work under my supervision and guidance and this project has not formed on the basis for the award of any Degree/ Diploma/ Associate Ship/ Fellowship or similar nature to any University/ Institution earlier.

Submitted for the viva-voce held on:

PLACE: Madurai

GUIDE

HEAD OF THE DEPARTMENT

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I hereby declare that the mini project titled "**Vibe Wave**" submitted to **SUBBALAKSHMI LAKSHMIPATHY COLLEGE OF SCIENCE**. I affirm that all information, data, and analysis presented in this report are authentic and have been collected and analyzed by me. I have also referred to and acknowledged all the sources, references, and materials used in preparing this project. It is a record of original work done by me and this project work has not formed the basis for the award of any degree / Associate ship /Fellowship or similar to any candidate in any university.

B.Sc. (Computer Science)
(Data Science and Analytics)

Yugesh A
Reg.No:242237

ACKNOWLEDGEMENT

Firstly, I am willing to express my heartfelt thanks to the Almighty for providing an excellent Opportunity and I also thank him for his blessings to be a successful human being in all walks of my life.

I would like to express my grateful thanks to our president **Dr.R. LAKSHMIPATHY** and our beloved principal **Dr.R. SUJATHA M.Sc., M.Phil., M.Tech., Ph.D.**, for permitting me to do this project and for her encouragement in my academic career.

I take this opportunity to convey thanks to my Head of the Department **Mrs.Anuradha. N M.Sc., M.Phil., NET.**, for her support to complete the project successfully and her guidance and moral support during the project period, because her keen interest and valuable suggestions have helped me throughout the project.

I would like to express my sincere gratitude and my thankfulness to my guide **Mrs.Yogeshwari .S M.Sc** for her guidance in the development of this **Vibe Wave** .I acknowledge the fullest support rendered by my faculty members. I am thankful to my friends who are the real supporters of my academic career. The knowledge and support I have received have been invaluable in completing this project successfully.

B.Sc. (Computer Science)
(Data Science and Analytics)

Yugesh A
(RegNo:242237)

TABLE OF CONTENT

S. No	Content	Page No
1.	Introduction 1.1 Abstract 1.2 Synopsis	1
2.	System Analysis 2.1 Existing System 2.2Proposed System	2
3.	System Requirement 3.1 Hardware Requirement 3.2 Software Requirement	4
4.	Modules Description	5
5.	Software Description	6
6.	APPENDIX 6.1Code Snippet 6.2Output Screenshot	9
7.	Conclusion	18
8.	Future Enhancement	19
9.	Bibliography	20

1.INTRODUCTION:

1.1. Introduction:

The domain of digital music streaming and storage has revolutionized how users access and manage their music collections. With the rise of cloud-based platforms, users no longer rely solely on local storage but instead utilize online services to store, stream, and manage their audio files.

Vibe Wave operates within this music management and streaming domain, offering a personalized cloud-based music library where users can upload, store, and retrieve their songs anytime, anywhere. The platform integrates web and mobile technologies with secure backend storage solutions to provide a seamless experience. With increasing demand for on-the-go music accessibility, data security, and personalized content, Vibe Wave addresses the need for a user-centric music management system that balances storage flexibility, ease of access, and affordability.

1.2. Synopsis:

Vibe Wave is a music management platform that enables users to upload, store, and access their songs seamlessly across multiple devices. Designed with a user-friendly interface, it offers both a web and mobile application for enhanced accessibility. Users can create profiles, upload their own songs, record samples, and download their music on any logged-in device. The platform ensures secure file management and authentication using Python and/or Node Js for backend operations and MongoDB for data storage. Vibe Wave follows a subscription-based model, providing different storage and feature options, including a free plan with limited storage and premium plans for expanded access. With a design inspired by leading music platforms, Vibe Wave aims to deliver a smooth and efficient music experience.

2.SYSTEM ANALYSIS:

2.1. Existing Systems:

Major platforms like Spotify, Apple Music, YouTube Music, SoundCloud dominate the music streaming market, each with unique strengths. While Spotify and Apple Music offer vast libraries and premium features, they lack user-upload options. YouTube Music integrates video streaming but includes ads in its free version. SoundCloud support independent artists but have limited mainstream content. Despite their strengths, these platforms lack universal cloud storage, flexible uploads, and built-in recording tools, which Vibe Wave aims to provide.

Drawbacks :

1. Limited User Control:

- ❖ Users cannot upload and store their own music for easy access across multiple devices.
- ❖ Music libraries are controlled by platforms, restricting personalization.

2. Lack of Personal Music Creation Features:

- ❖ No option to record, save, or manage personal audio samples for future song creation.
- ❖ Users rely on external software for recording and storing their own music.

3. Storage Limitations & Expensive Premium Plans

- ❖ Free plans offer very limited storage, restricting user uploads.
- ❖ Paid plans are often expensive with no affordable mid-tier options.

4. Limited Cross-Platform Accessibility

- ❖ Many platforms do not allow seamless switching between web and mobile applications.
- ❖ Users often face restrictions on accessing their music on multiple devices.

2.2 Proposed System:

Vibe Wave is a music storage and streaming platform designed to give users complete control over their personal music collections. Unlike traditional streaming services Vibe Wave allows users to upload, manage, and access their own songs anytime, anywhere eliminating the reliance on third-party cloud storage or external applications.

Advantages:

1. Personalized Music Storage

- ❖ Users can upload, store, and access their own music files from any device.
- ❖ Provides a dedicated cloud storage system for personal music collections.

2. Offline Access & Seamless Playback

- ❖ Users can download their uploaded songs for offline listening without restrictions.
- ❖ Music can be played anytime, anywhere, without requiring an internet connection.

3. Built-in Recording & Sample Storage

- ❖ Enables users to record, save, and manage their own audio samples.
- ❖ Helps aspiring musicians store ideas and create their own tracks.

4. Cross-Platform Accessibility

- ❖ Fully functional on both web and mobile platforms for seamless music access.
- ❖ Users can switch between devices without losing access to their music.

5. Secure & Encrypted File Storage

- ❖ Ensures user data and uploaded music are securely stored and protected.
- ❖ Uses authentication systems to prevent unauthorized access.

3. SYSTEM REQUIREMENTS:

3.1. Software Requirements:

Front-end

1. Web Browser: Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge
2. JavaScript Framework: React, Node
3. CSS Framework: Bootstrap or Tailwind CSS
4. HTML5: For structuring and presenting content

Back-end

1. Programming Language: Node.js, Python.
2. Database: MongoDB

3.2. Hardware Requirements:

Server

1. Processor: Quad-core or higher
2. Memory: 8 GB RAM or higher
3. Storage: 256 GB SSD or higher
4. Operating System

Client-side

1. Device: Desktop, laptop, tablet, or smartphone
2. Processor: Dual-core or higher
3. Memory: 4 GB RAM or higher
4. Storage: 128 GB or higher
5. Operating System: Windows, macOS, or mobile OS (Android or iOS)

4.SOFTWARE DESCRIPTION:

4.1. Frontend (User Interface)

The frontend is responsible for the user interaction and experience. It includes:

✓ Technologies Used: HTML, CSS, JavaScript.

✓ Key Features:

- ❖ User-friendly dashboard for study tracking.
- ❖ Interactive goal-setting and study session planner.
- ❖ Leaderboards and gamification for motivation.
- ❖ Responsive design for compatibility across devices.

4.2. Backend (Server-Side Logic)

The backend processes user requests, manages data, and ensures smooth system functionality.

✓ Technology Used: Node.js with Express.js.

✓ Key Features:

- ❖ Handles user authentication.
- ❖ Manages study schedules, progress tracking, and reminders.
- ❖ Processes leaderboard rankings and achievement tracking.
- ❖ Ensures secure communication between frontend.

4.3. Database (Data Storage & Management)

The database stores all user study plans, progress, and leaderboard rankings securely.

✓ Technology Used: MongoDB

✓ Key Features:

- ❖ Stores user profiles, study schedules, and activity logs
- ❖ Ensures quick data retrieval for seamless study tracking
- ❖ Provides structured data storage for scalability

5.SYSTEM DESCRIPTION:

5.1. User Authentication Module

❖ **Purpose:** Manages user registration, login, and authentication.

❖ **Features:**

- User sign-up with email and password.
- Secure login with encrypted credentials.
- Session management (remember me, logout).
- Password recovery/reset functionality.

Technologies Used: Python (Flask/Django), MongoDB, JWT authentication.

5.2. User Profile & Account Management Module

❖ **Purpose:** Allows users to update and manage their profile details.

❖ **Features:**

- View and update profile information.
- Track uploaded songs & storage usage.

Technologies Used: MongoDB, JavaScript (frontend).

5.3. Music Upload & Storage Module

❖ **Purpose:** Enables users to upload and manage their personal music files.

❖ **Features:**

- Upload MP3, WAV, FLAC files.
- Secure file encryption and retrieval.

Technologies Used: Python (Flask/Django), MongoDB.

5.4. Music Streaming & Playback Module

❖ **Purpose:** Provides a seamless music playback experience.

❖ **Features:**

- Play, pause, rewind, and fast-forward controls.
- Adaptive streaming based on internet speed.
- Playlist creation and shuffle options.

Technologies Used: JavaScript (Frontend Player), Web Audio API.

5.5. User Dashboard Module

❖ **Purpose:** Displays user data and activity in an interactive dashboard.

❖ **Features:**

- Recent uploads and most played songs.
- Storage usage statistics.

Technologies Used: JavaScript, Chart.js (for analytics).

5.6. Search & Music Discovery Module

❖ **Purpose:** Helps users find songs quickly.

❖ **Features:**

- Search bar for song name, artist, or album.
- Filters (e.g., recently played, favorite songs).

Technologies Used: JavaScript, MongoDB queries, Elasticsearch (future enhancement).

5.7. File Security & Access Control Module

❖ **Purpose:** Ensures secure storage and controlled access to music files.

❖ **Features:**

- End-to-end encryption for uploaded files.
- Access control (only account owner can download).
- Data backup & recovery options (future enhancement).

Technologies Used: Python (Flask/Django), AWS S3/GCP Storage.

5.9. Admin & Analytics Module

❖ **Purpose:** Provides administrative control and system monitoring.

❖ **Features:**

- User management.
- System logs & usage analytics.
- Content moderation for uploaded files.

Technologies Used: Python (Flask/Django), MongoDB, Admin Dashboard Framework.

6.APPENDIX:

6.1. CODE SNIPPET :

```
<!DOCTYPE html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

    <meta http-equiv="X-UA-Compatible" content="IE=edge" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <!--
      - primary meta tags
    -->

    <title>Music Player</title>

    <meta name="title" content="Music Player" />

    <meta
      name="description"
      content="A web music player html template made by codewithsadee" />

    <!--
      - favicon
    -->

    <link rel="shortcut icon" href="/favicon.svg" type="image/svg+xml" />

    <!-- - google font link-->

    <link rel="preconnect" href="https://fonts.googleapis.com" />

    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
```

```

<link
  href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500&display=swap"
  rel="stylesheet"
/>

<!--
- material icon font
-->

<link
  rel="stylesheet"
  href="https://fonts.googleapis.com/css2?family=Material+Symbols+Rounded:opsz,wght,FILL,GRAD
@24,400,0,-25"
/>

<!--
- custom css link
-->

<link rel="stylesheet" href="dashboard.css" />

</head>

<body>

<!--
- #TOP APP BAR
-->

<div class="top-bar wrapper">

  <div class="logo wrapper">

    <span class="material-symbols-rounded">graphic_eq</span>

```

```

    <h1 class="title-lg">Music Player</h1>

</div>

</div>

<main>

  <article>

    <!--

    - #PLAYER

    -->

    <div class="player">

      <div class="container">

        <figure class="music-banner">

        </figure>

        <div class="music-content">

          <h2 class="headline-sm" data-title>Happy Moments (Master)</h2>

          <p class="label-lg label-wrapper wrapper">

```



```
<span data-album>No Spirit</span>
```

```
<span data-year>2022</span>
```

```
</p>
```

```
<p class="label-md artist" data-artist>No Spirit x Tonion</p>
```

```
<div class="seek-control">
```

```
<div>
```

```
<div class="range-wrapper">
```

```
<input
```

```
  type="range"
```

```
  step="1"
```

```
  max="60"
```

```
  value="0"
```

```
  class="range"
```

```
  data-range
```

```
  data-seek
```

```
<div class="range-fill" data-range-fill></div>
```

```
</div>
```

```
<div class="duration-label wrapper">
```

```
<span class="label-md" data-running-time>0:00</span>
```

```
<span class="label-md" data-duration>1:00</span>
```

```
</div>
```

```
</div>
```

```

<div class="volume">

  <button class="btn-icon" data-volume-btn>

    <span class="material-symbols-rounded">volume_up</span>

  </button>

  <div class="range-wrapper">

    <input

      type="range"

      step="0.05"

      max="1"

      value="1"

      class="range"

      data-range

      data-volume

    />

    <div class="range-fill" data-range-fill></div>

  </div>

</div>

<div class="player-control wrapper">

  <button class="btn-icon toggle" data-repeat>

    <span class="material-symbols-rounded default-icon"

      >repeat</span>

    >

    <span class="material-symbols-rounded active-icon" >repeat_one</span>

```

```
</button>

<button class="btn-icon" data-skip-prev>

  <span class="material-symbols-rounded">skip_previous</span>

</button>

<button class="btn-icon play" data-play-btn>

  <span class="material-symbols-rounded default-icon"

    >play_arrow</span

  >

  <span class="material-symbols-rounded active-icon"

    >pause</span>

</button>

<button class="btn-icon" data-skip-next>

  <span class="material-symbols-rounded">skip_next</span>

</button>

<button class="btn-icon toggle" data-shuffle>

  <span class="material-symbols-rounded">shuffle</span>

</button>

</div>

</div>

</div>

</div>

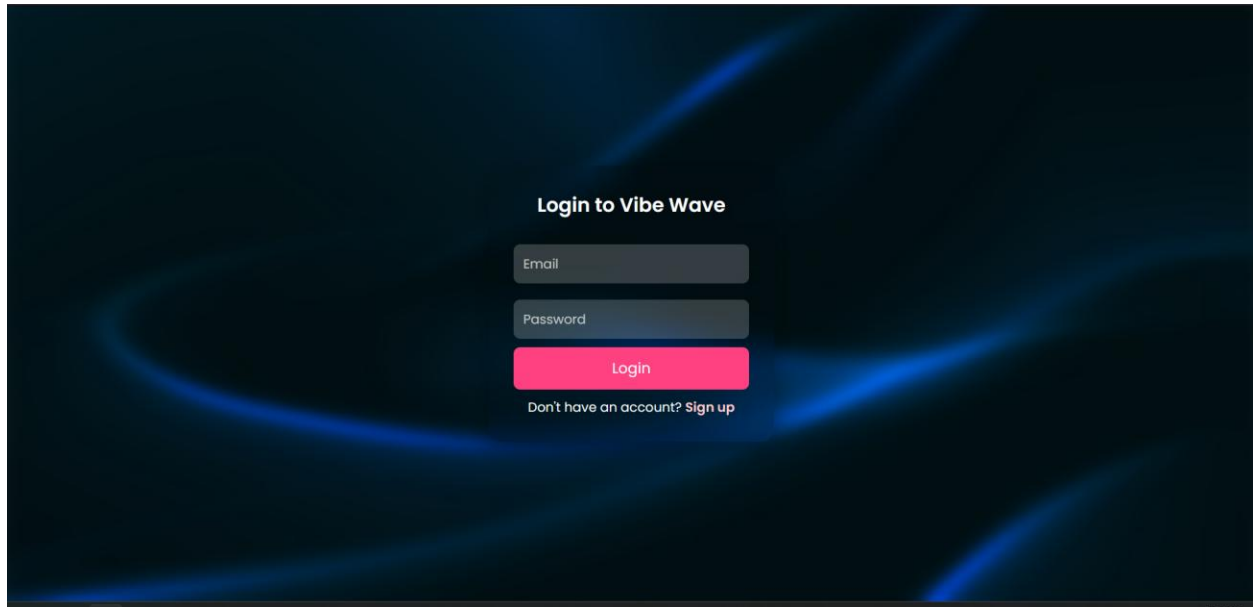
<script src="dashboard.js"></script>

</body>

</html>
```

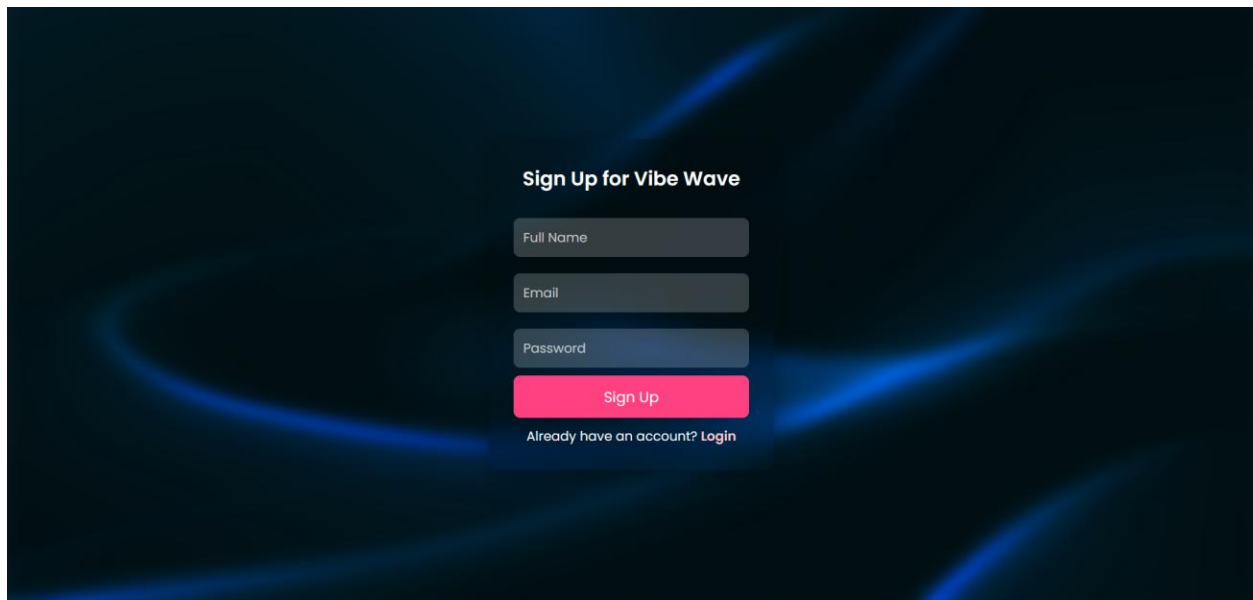
6.2. OUTPUT:

6.2.1. LOGIN PAGE:



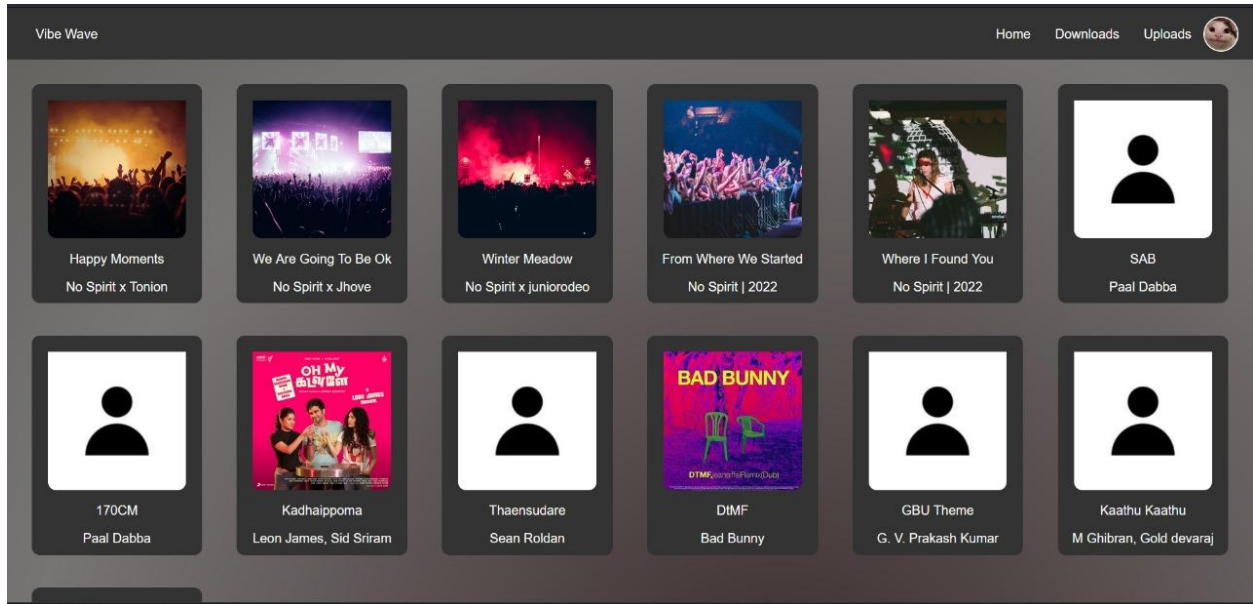
The login page features a dark blue background with abstract, glowing blue wave patterns. In the center, there is a white text label "Login to Vibe Wave". Below this label are two input fields: "Email" and "Password", both with light gray borders and placeholder text. A prominent pink "Login" button is positioned below the password field. At the bottom of the form, there is a link that reads "Don't have an account? Sign up".

6.2.2. SIGNUP PAGE:

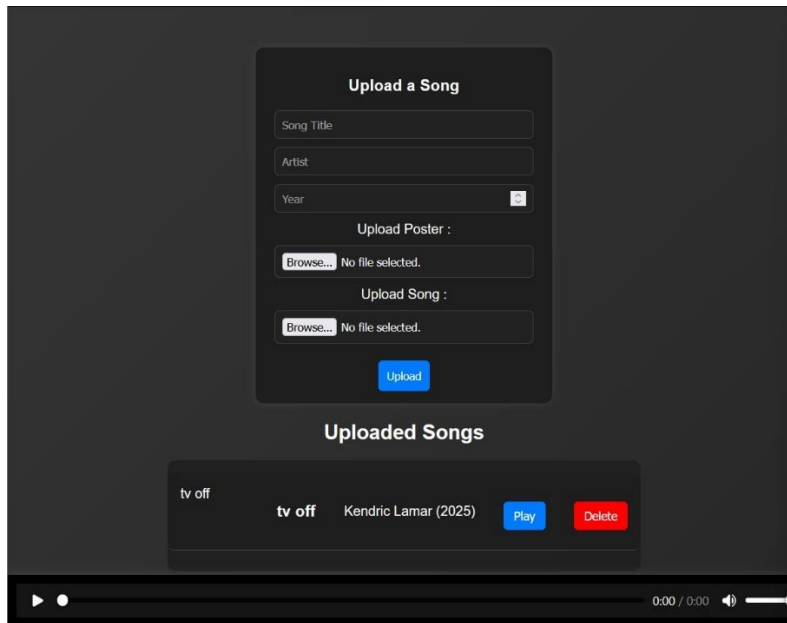


The sign up page has the same dark blue background with glowing blue wave patterns as the login page. The central text label reads "Sign Up for Vibe Wave". Below this are three input fields: "Full Name", "Email", and "Password", each with a light gray border and placeholder text. A pink "Sign Up" button is located below the password field. At the bottom, there is a link that says "Already have an account? Login".

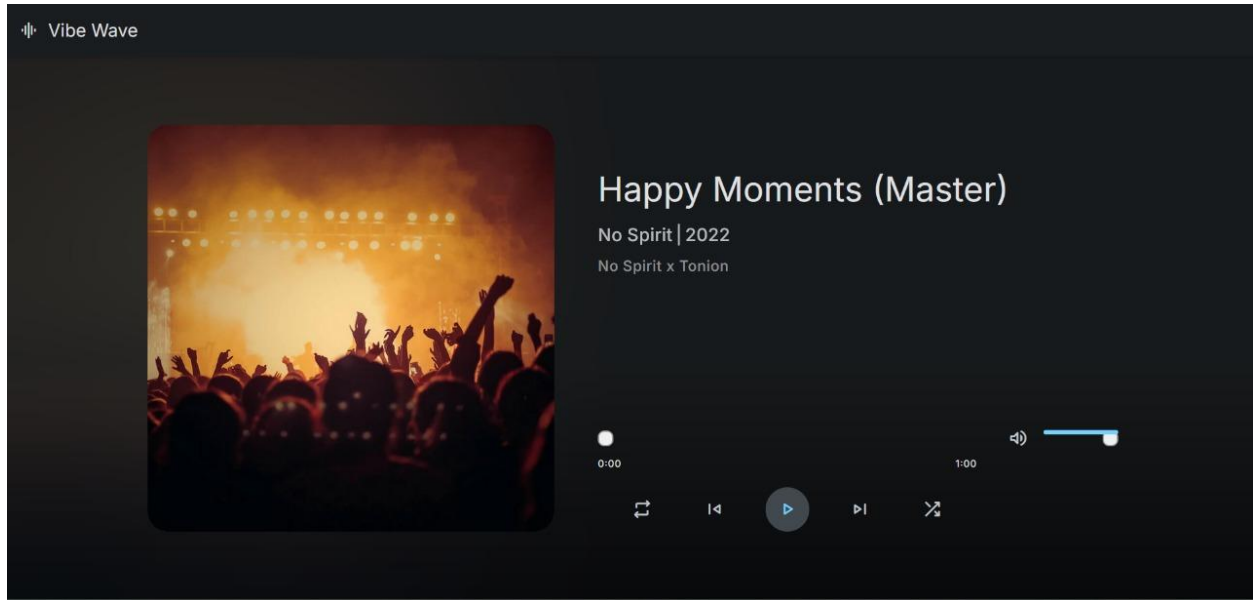
6.2.3. DASHBOARD:



6.2.4. UPLOADING SONGS:



6.2.5. MUSIC PLAYER:



7. CONCLUSION:

Vibe Wave is a powerful and user-friendly music platform designed to give users complete control over their personal music collection. By integrating secure authentication, cloud storage, and seamless playback, it ensures easy access to music across devices. The project demonstrates the efficient use of web technologies like HTML, CSS, JavaScript, Python, and MongoDB to create a smooth and interactive user experience.

With features like song uploads, downloads, and a built-in recording studio, Vibe Wave provides a versatile solution for music enthusiasts who want to store and manage their songs without relying on third-party streaming services. This mini-project not only showcases technical expertise in full-stack development but also highlights the potential for future enhancements, such as AI-powered recommendations or collaborative playlists.

Overall, Vibe Wave is a step toward a more personalized and independent music experience, offering a secure and accessible platform for users to enjoy their music anytime, anywhere.

8.FUTURE ENHANCEMENT:

The Vibe Wave has great potential for further development to improve user experience and functionality. Some of the future enhancements include:

1. AI-Powered Recommendations

Implement AI-based music recommendations based on user listening history and preferences.

Provide personalized playlists and mood-based suggestions using ML algorithms.

2. Collaborative Playlists

Allow multiple users to create, edit, and share playlists with friends.

Enable real-time playlist collaboration, similar to Spotify Blend.

3. Social & Community Features

Introduce a social feed where users can follow others, like, comment, and share songs.

Enable artist profiles where independent artists can showcase their work.

4. Offline Mode & Smart Downloads

Allow offline playback for downloaded songs.

Implement a smart download feature that automatically downloads favorite songs when on Wi-Fi.

5. Advanced Audio Settings

Add an equalizer to customize sound preferences.

Provide high-quality streaming options (e.g., FLAC or 320kbps MP3).

6. Live Streaming & Virtual Concerts

Enable artists to live-stream performances directly from the platform.

Host virtual concerts and fan meetups with ticketed access.

9.BIBLIOGRAPHY:

Websites & Online Documentation

- MongoDB Official Documentation – <https://www.mongodb.com/docs/>
- Node.js Official Documentation – <https://nodejs.org/en/docs>
- Express.js Guide – <https://expressjs.com/>
- W3Schools(Frontend & Backend Technologies) – <https://www.w3schools.com/>

Additional Learning Resources

- GitHub Repositories – Various open-source projects and study planner applications as references.
- Stack Overflow – Community discussions and troubleshooting solutions.